

ALERT-02: Choosing and Building Detection

Series: ALERT — Alerting Strategy and Design | **Notebook:** 02 of 05 | **Created:** June 2026 | **Last Updated:** 06/16/2026

Overview

Detection is a choice between four mechanisms, and picking the wrong one is the root of most alert noise. This notebook is the **decision framework** — which mechanism for which signal, and why — then it hands off to AIOPS-02 for the build mechanics and SLO-04 for reliability alerting. It deliberately does not re-document the anomaly detector's knobs; it tells you *which* tool to reach for.

Table of Contents

- [1. The Four Mechanisms](#)
 - [2. The Decision](#)
 - [3. Anti-Patterns](#)
 - [4. Prototype Before You Commit](#)
 - [5. Hand-Off](#)
-

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS Gen3 with the Anomaly Detection app
Prior reading	ALERT-01 (the end-to-end picture)
Build mechanics	AIOPS-02 (anomaly detector setup), SLO-02/04 (SLIs and burn-rate)

1. The Four Mechanisms

Mechanism	What it detects	Owns the detail
OOTB Davis	Latency/error/saturation anomalies, automatically, with seasonal baselines	AIOPS-02 §1
Custom Davis anomaly detector	A business-specific signal Davis does not cover, expressed as a DQL query an analyzer judges	AIOPS-02 §4
OpenPipeline-derived metric	A signal that lives in logs/spans, extracted to a metric at ingest and then alerted on cheaply	OPIPE, AIOPS-02 §6
SLO burn-rate	A user journey burning its error budget too fast	SLO-04

These are not alternatives to rank once — they are a toolkit. Most environments use all four.

2. The Decision

Walk it top-down and stop at the first match:

1. **Is OOTB Davis already detecting it?** Check the problem feed and anomaly detection settings first. If yes → tune sensitivity, do not build anything.
2. **Is it a reliability promise about a user journey?** → an **SLO** with burn-rate alerting (SLO-04). This is the highest-signal option.

3. **Does the signal live in logs or spans, and recur?** → extract an **OpenPipeline metric**, then put a detector on the metric. Cheaper than querying raw data on every evaluation.
4. **Is it a custom condition on existing metrics?** → a **custom Davis anomaly detector** with an auto-adaptive or seasonal analyzer.

The cost — to build and to maintain — rises as you go down. Staying high is the lever on noise (ALERT-01 §2).

3. Anti-Patterns

- **Static thresholds on traffic-correlated metrics.** They alert every off-peak hour and every traffic spike. Use auto-adaptive or seasonal (AIOPS-02 §1). Reserve static thresholds for true hard limits (SLO/contract/capacity).
- **Duplicating Davis.** Before building a custom detector, confirm OOTB Davis or an existing metric event does not already cover the condition. Duplicate detection means duplicate alerts.
- **Querying logs/traces directly in a recurring detector.** Pays query cost on every evaluation, forever. Extract a metric first (FAQ-09, OPIPE).
- **A detector with a bare event template.** No team/zone property means nothing to route on (ALERT-01 §4).

4. Prototype Before You Commit

Whichever mechanism you choose, develop the underlying query in a notebook first — run it, confirm it returns a sane result over a representative window, *then* wire it into the detector or SLO. A detector built on a query that silently returns nothing never fires and is worse than no alert, because the team believes it is covered. This notebook-as-scratchpad discipline is covered in AIOPS-02 §4 and SLO-02 §6.

5. Hand-Off

Mechanism chosen	Build it in
Tune OOTB Davis	AIOPS-02 §1–§3

Mechanism chosen	Build it in
Custom anomaly detector	AIOPS-02 §4 (analyzer, tuning knobs, event template)
Metric events / schemas as code	AIOPS-02 §6, AUTOM-05/06
OpenPipeline-derived metric	OPIPE, FAQ-09
SLO burn-rate	SLO-02 (SLI), SLO-04 (alerting)

Then return to ALERT-03 to route what fires.

Sources: [Anomaly Detection app \(DT docs\)](#). **Derived:** the top-down decision order synthesises OOTB-first guidance with the query-cost economics in OPIPE/FINOPS.

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.